Python — datová analýza v Pandas (PYDATA1)

Úvod do problematiky a nástrojů

Petr Gregor

ICT Pro

9.-10. března 2026

Petr Gregor

- Lektor
  - Python
  - Java
  - AI
  - Data Science
- Analytik na MUNI
- Programátor (Pascal, C, C++, C#, Java, JavaScript, PHP, VB, Python, Ruby,. . .)
- 25+ let zkušeností s výukou
- 20+ let zkušeností s programováním

Program kurzu (1/2)

1. Nástroje datové analýzy
   - Formulace problému
   - Statistické metody
2. Jupyter notebook
   - Instalace a spuštění
   - Text a kód v jednom dokumentu
   - Spouštění jednotlivých kroků
   - Rozdíly od běžné práce v Pythonu
3. Datové nástroje
   - Pandas a související knihovny
   - DataFrame a Series
   - Zpracování tabulkových dat
4. Zdroje dat
   - Tabulkové formáty (Excel, CSV)
   - Databázové zdroje (SQL)
   - Webové zdroje (webscraping)

Program kurzu (2/2)

5. Zpracování dat
   - Transformace tabulkových dat
   - Filtrování řádků a sloupců
   - Konverze datových typů
   - Agregace dat
6. Statistické nástroje
   - Statistické vlastnosti dat
   - Ověřování korelací
7. Výstupy a prezentace výsledků
   - Tabulky a grafy
   - Datové formáty a SQL
8. Kam pokračovat dál
   - Další knihovny a nástroje
   - Možnosti strojového učení

Nástroje datové analýzy

Proč zvolit Python a Pandas?

- **Efektivita:** Rychlejší než ruční zpracování v Excelu.
- **Kontrola:** Plná kontrola nad každým krokem transformace dat.
- **Automatizace:** Skripty lze opakovaně spouštět na nová data.
- **Škálovatelnost:** Práce s velkými soubory, které Excel nezvládne.

Cíl kurzu

Přejít od prostého prohlížení tabulek k aktivnímu programovému zpracování a vizualizaci dat.

Formulace problému

Než začneme psát kód, musíme definovat analytický záměr:

- **Definice otázky:** Co přesně chceme v datech najít?
- **Hypotéza:** Předpoklad, který chceme potvrdit nebo vyvrátit (např. „Sleva zvýšila obrat, ale snížila celkový zisk“).
- **Typy analýzy:**
  - *Deskriptivní:* Co se stalo?
  - *Prediktivní:* Co se pravděpodobně stane?
- **Sběr dat:** Identifikace zdrojů (interní databáze, CSV, webscraping).

Statistické metody: Deskriptivní statistika

Základní nástroje pro pochopení struktury dat:

- **Míry polohy:**
  - **Průměr (Mean):** Citlivý na extrémní hodnoty.
  - **Medián:** Robustní střední hodnota (odolná vůči outlierům).
  - **Modus:** Nejčastější hodnota v datech.
- **Míry variability:**
  - **Rozptyl a Směrodatná odchylka:** Jak moc jsou data „rozlítaná“ od průměru.
  - **Rozsah (Range):** Rozdíl mezi maximem a minimem.

Statistické metody: Distribuce a vztahy

Pohled pod povrch tabulkových dat:

- **Distribuce dat:** Normální rozdělení vs. zkosená data.
- **Outliery (Odlehlé hodnoty):** Jak identifikovat anomálie, které mohou zkreslit výsledek.
- **Korelace vs. kauzalita:**
  - Sledují dvě proměnné stejný trend? (Korelace).
  - Způsobuje jedna proměnná druhou? (Kauzalita).

Zlaté pravidlo

Korelace neznamená příčinnou souvislost (kauzalitu).

Jupyter notebook

Co je Jupyter Notebook?

- **Interaktivní prostředí:** Umožňuje spouštět bloky kódu nezávisle na sobě.
- **Literate Programming:** Kombinace živého kódu, textu (Markdown), rovnic a vizualizací.
- **Webové rozhraní:** Běží v prohlížeči, ale kód se vykonává v lokálním nebo vzdáleném *kernelu*.
- **Standard v Data Science:** Ideální pro experimentování a dokumentaci postupu.

Instalace a spuštění

Nejjednodušší cestou je instalace přes `pip` nebo distribuce jako **Anaconda**.

```
# Install Jupyter using pip
pip install jupyter

# Launch the notebook server
jupyter notebook

# Launch JupyterLab interface
jupyter lab
```

Tip pro kurz

V rámci kurzu budeme používat modernější rozhraní **JupyterLab**, které spustíte příkazem `jupyter lab`.

Buňky: kód vs. text

Notebook se skládá z buněk (*cells*):

1. **Code cells:** Obsahují spustitelný Python kód. Výstup se zobrazí přímo pod buňkou.
2. **Markdown cells:** Slouží k popisu. Podporují:
   - Nadpisy (#, ##)
   - Odrážky a tučný text
   - Formátování kódu a LaTeX rovnice

Základní klávesové zkratky

Efektivita v Jupyteru závisí na znalosti zkratek (v *Command módu* - modrý okraj):

- `Shift + Enter`: Spustí buňku a vybere následující.
- `A` / `B`: Vloží novou buňku nad (*Above*) nebo pod (*Below*).
- `D + D`: Odstraní vybranou buňku.
- `M` / `Y`: Přepne buňku na Markdown / Kód.

Specifika oproti běžnému Pythonu

- **State persistence:** Proměnné zůstávají v paměti i po smazání buňky (pokud nerestartujete kernel).
- **Out-of-order execution:** Pozor na spouštění buněk „napřeskáčku“ – může vést k nekonzistentním výsledkům.
- **Magické příkazy (Magics):** Speciální příkazy začínající %.

```
# List all defined variables
%who

# Measure execution time of a cell
%%time
data_list = [x for x in range(1000000)]
```

Datové nástroje

Co je to Pandas?

- **PANel DAta:** Název odvozen z ekonometrického termínu pro multidimenzionální data.
- **Cíl:** Efektivní manipulace s tabulkovými daty (podobně jako v SQL nebo Excelu).
- **Základ:** Postaveno na knihovně `NumPy`, což zajišťuje vysoký výkon při operacích s poli.

Importování knihovny

Standardní konvence v Python komunitě: `import pandas as pd`

Struktura: Series

Series je jednorozměrné pole, které může obsahovat jakýkoliv datový typ. Každý prvek má svůj index.

```
import pandas as pd

# Creating from a list
data = [10, 20, 30, 40]
prices = pd.Series(data, name='Price') # Simple numeric series

print(prices)
# Output includes Index and Values
```

Struktura: DataFrame

DataFrame je dvourozměrná tabulka. Můžeme si ji představit jako kolekci objektů `Series`, které sdílejí stejný index.

```
# Creating from a dictionary
student_data = {
    'name': ['Alice', 'Bob', 'Charlie'],
    'grade': [1, 2, 1]
}

df = pd.DataFrame(student_data) # Global variable df
print(df.head()) # Show first 5 rows
```

Indexování a výběr dat

Pandas nabízí dva hlavní způsoby, jak přistupovat k datům:

- `iloc`: Indexování pomocí celých čísel (pozice).
- `loc`: Indexování pomocí jmen (labelů).

```
# Select first row
first_row = df.iloc[0]

# Select by column name
names = df['name']

# Filter students with grade 1
top_students = df[df['grade'] == 1] # Boolean indexing
```

Základní inspekce dat

Před analýzou musíme vědět, co v datech je:

- `df.info()`: Přehled datových typů a chybějících hodnot.
- `df.describe()`: Základní statistický souhrn (průměr, min, max).
- `df.shape`: Rozměry tabulky (řádky, sloupce).

```
# Get statistical summary for numeric columns
stats = df.describe()
```

Zdroje dat

Široká podpora formátů

Knihovna Pandas je vybavena robustními funkcemi pro čtení různých formátů:

- **Textové soubory:** CSV, TSV, JSON, XML.
- **Binární formáty:** Excel (XLSX, XLS), Parquet, HDF5.
- **Databáze:** SQL (přes SQLAlchemy).
- **Web:** HTML tabulky, API volání.

Práce s CSV a Excel soubory

CSV je nejčastější formát. Excel vyžaduje doplňkovou knihovnu `openpyxl`.

```
# Read a CSV file with specific delimiter
df_csv = pd.read_csv('data.csv', sep=';', encoding='utf-8')

# Read specific sheet from Excel
df_excel = pd.read_excel('report.xlsx', sheet_name='
    Sales_2023')

# Quick preview
print(df_csv.head(3)) # Show first 3 rows
```

Optimalizace importu: Načítání vybraných sloupců

U rozsáhlých datasetů je neefektivní načítat všechny sloupce, pokud jich potřebujeme jen několik. Parametr `usecols` umožňuje specifikovat výběr již při čtení.

- **Seznam názvů:** Přesná specifikace jmen sloupců.
- **Indexy:** Výběr pomocí pořadového čísla sloupce (např. 0, 2, 5).
- **Lambda funkce:** Dynamický výběr (např. všechny sloupce začínající na 'ID').

```
# 1. Selection by names
needed_cols = ['order_id', 'customer_id', 'total_price']
df = pd.read_csv('big_data.csv', usecols=needed_cols)

# 2. Selection by index (0 is first column)
df_small = pd.read_csv('big_data.csv', usecols=[0, 1, 5])

# 3. Selection by logic (e.g. skip internal columns)
df_clean = pd.read_csv(
    'big_data.csv',
    usecols=lambda x: x not in ['internal_log', 'debug_info']
```

Načítání z SQL databází

Pro SQL využíváme Pandas v kombinaci s sqlalchemy. To umožňuje univerzální připojení k SQLite, PostgreSQL, MySQL nebo MS SQL.

```
from sqlalchemy import create_engine

# Create a connection engine (example for SQLite)
engine = create_engine('sqlite:///my_database.db')

# Load data using a SQL query
query = "SELECT * FROM orders WHERE status = 'shipped'"
df_sql = pd.read_sql(query, engine) # Load into DataFrame
```

Webscraping: Extrakce z HTML

Pandas dokáže jedním příkazem extrahovat všechny tabulky (`<table>`) z webové stránky.

```
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP'

# Returns a list of all DataFrames found on the page
tables = pd.read_html(url)

# Access the first table
gdp_table = tables[0]
```

Upozornění

Metoda `read_html` vyžaduje knihovny `lxml` nebo `beautifulsoup4`.

Zápis dat (export)

Stejně snadno, jako data čteme, je můžeme i ukládat pro další použití.

```
# Save to CSV without index column
df.to_csv('cleaned_data.csv', index=False)

# Save to Excel
df.to_excel('output.xlsx', index=False)

# Save back to SQL table
df.to_sql('results', engine, if_exists='replace')
```

Zpracování dat

Čištění dat: Proč je to nutné?

Data z reálného světa jsou málokdy dokonalá. Časté problémy:

- **Chybějící hodnoty:** Prázdná pole (`NaN` – Not a Number).
- **Duplicity:** Identické řádky vzniklé chybou při sběru.
- **Špatné typy:** Čísla uložená jako text, data v nečitelném formátu.
- **Odlehlé hodnoty:** Extrémní chyby v měření.

Práce s chybějícími hodnotami

Pandas nabízí tři základní strategie pro řešení `NaN` hodnot:

1. **Odstranění:** `df.dropna()`
2. **Výplň konstantou:** `df.fillna(0)`
3. **Imputace:** Nahrazení průměrem či mediánem.

```
# Check for null values
print(df.isnull().sum())

# Drop rows where 'price' is missing
df_clean = df.dropna(subset=['price'])

# Fill missing 'category' with a default value
df['category'] = df['category'].fillna('Unknown')
```

Transformace sloupců

Vytváření nových metrik a úprava stávajících.

```
# Basic math operations
df['total_price'] = df['quantity'] * df['unit_price']

# String operations (using .str accessor)
df['email'] = df['email'].str.lower()

# Converting data types
df['date'] = pd.to_datetime(df['date'])
df['age'] = df['age'].astype(int)
```

Mapování a nahrazování

Změna hodnot na základě slovníku nebo funkce.

```
# Replace specific values
status_map = {'S': 'Shipped', 'P': 'Pending', 'C': 'Canceled'
    }
df['status_full'] = df['status'].map(status_map)

# Custom function application
def classify_temp(temp):
    return 'Hot' if temp > 25 else 'Cold'

df['weather'] = df['temperature'].apply(classify_temp)
```

Filtrování a výběr podmnožin

Kombinování logických podmínek pro získání konkrétních dat.

```
# Multiple conditions: AND (&), OR (|)
# Filter: Sales in Prague with volume > 100
filtered_df = df[(df['city'] == 'Prague') & (df['volume'] > 100)]

# Using .query() for readable syntax
expensive_items = df.query('price > 1000')
```

Pozor

Při kombinování podmínek v Pandas vždy používejte kulaté závorky kolem jednotlivých výrazů.

Výběr a filtrování sloupců

V Pandas existuje několik způsobů, jak vybrat pouze relevantní data a snížit tak paměťovou náročnost.

- **Výběr jednoho sloupce:** Výsledkem je objekt `Series`.
- **Výběr více sloupců:** Musíme použít seznam názvů (double brackets).
- **Metoda `drop`:** Odstranění nepotřebných sloupců.

```
# Selecting a single column (returns Series)
names = df['product_name']

# Selecting multiple columns (returns DataFrame)
subset = df[['product_name', 'selling_price']]

# Dropping unnecessary columns
# axis=1 specifies columns, inplace=True modifies the original DF
df.drop(['buying_price', 'internal_id'], axis=1, inplace=True)
```

Dynamický výběr sloupců

Větší datasety vyžadují automatizovaný přístup založený na vzorcích nebo typech dat.

```
# Select columns by data type (e.g., only numbers)
numeric_df = df.select_dtypes(include=['number'])

# Select columns using string methods (e.g., all starting
    with 'sales_')
sales_columns = [col for col in df.columns if col.startswith(
    'sales_')]
df_sales = df[sales_columns]
```

Dobrá praxe

Při práci s velkými daty je lepší vybrat potřebné sloupce hned při načítání pomocí parametru `usecols` ve funkci `read_csv()`.

Konverze datových typů

Správný datový typ je základem pro efektivní analýzu. Pandas umožňuje měnit typy pomocí metody `astype()` nebo specializovaných funkcí.

- **Číselné typy:** Převod textu na `int` nebo `float`.
- **Logické hodnoty:** Převod na `bool` (True/False).
- **Kategorické data:** Typ `category` pro opakující se textové hodnoty (šetří až 90 % paměti).

```
# Convert price to float and quantity to integer
df['price'] = df['price'].astype(float)
df['quantity'] = df['quantity'].astype(int)

# Convert a column to categorical data
df['region'] = df['region'].astype('category')
```

Práce s daty (DateTime)

Textové řetězce reprezentující datum je nutné převést na typ `datetime`, abychom mohli využívat časové operace.

```
# Simple conversion
df['order_date'] = pd.to_datetime(df['order_date'])

# Conversion with specific format (faster and safer)
df['delivery_date'] = pd.to_datetime(
    df['delivery_date'],
    format='%d.%m.%Y' # Czech date format
)

# Extracting components
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month_name()
```

Řešení chyb při konverzi

Často se v datech vyskytují chyby (např. text "N/A" v číselném sloupci), které standardní konverzi přeruší.

```
# Parameter 'errors' can be: 'raise', 'coerce', 'ignore'
# 'coerce' will turn invalid values into NaN
df['price'] = pd.to_numeric(df['price'], errors='coerce')

# Identifying rows that failed to convert
failed_rows = df[df['price'].isna()]
```

Tip pro lektora

Ukažte účastníkům rozdíl v paměti (`df.memory_usage()`) před a po konverzi sloupce s městy na typ `category`.

Princip Agregace: Split – Apply – Combine

Agregace dat v Pandas obvykle probíhá ve třech fázích:

1. **Split (Rozdělení):** Data jsou rozdělena do skupin na základě určitého klíče (např. Kategorie).
2. **Apply (Aplikace):** Na každou skupinu je aplikována funkce (např. součet, průměr).
3. **Combine (Spojení):** Výsledky jsou spojeny do nové datové struktury.

Základní seskupování: groupby()

Nejčastější způsob agregace pomocí metody groupby.

```
# Sum of sales per category
sales_sum = df.groupby('category')['revenue'].sum()

# Multiple statistics for a single column
sales_stats = df.groupby('category')['revenue'].agg(['mean',
    'sum', 'count'])

# Grouping by multiple columns
monthly_category_sales = df.groupby(['year', 'category'])['
    revenue'].sum()
```

Pokročilá agregace a transformace

Někdy potřebujeme provést různé výpočty pro různé sloupce nebo zachovat původní počet řádků.

```
# Different functions for different columns
custom_agg = df.groupby('category').agg({
    'revenue': 'sum',
    'quantity': 'mean',
    'customer_id': 'nunique' # Count unique customers
})

# Broadcast the group mean back to the original rows
# Useful for normalization or finding outliers
df['category_avg'] = df.groupby('category')['price'].
    transform('mean')
```

Kontingenční tabulky: pivot_table()

Pro multidimenzionální pohled na data jsou ideální kontingenční tabulky, podobné těm v Excelu.

```
# Sales by Year (rows) and Category (columns)
pivot = df.pivot_table(
    values='revenue',
    index='year',
    columns='category',
    aggfunc='sum',
    fill_value=0 # Replace NaNs with zeros
)
```

Rozdíl mezi GroupBy a Pivot

`groupby` je obvykle rychlejší a vrací hierarchický index, zatímco `pivot_table` vytváří tabulku vhodnější pro vizuální srovnání a export.

Statistické nástroje

Rychlý statistický přehled

Pandas poskytuje vestavěné metody pro okamžitý výpočet deskriptivní statistiky celého datasetu.

- `describe()`: Základní souhrn (počet, průměr, odchylka, kvartily).
- `mean()`, `median()`, `mode()`: Míry centrální tendence.
- `std()`, `var()`: Míry rozptýlenosti (variability).

```
# Summary for all numeric columns
summary = df.describe()

# Specific calculation for one column
average_price = df['price'].mean()
middle_value = df['price'].median() # Robust to outliers
```

Kvantily a rozdělení dat

Kvantily nám pomáhají pochopit, jak jsou data rozprostřena a identifikovat odlehlé hodnoty (outliery).

- **Kvartily:** Rozdělují data na čtvrtiny (25 %, 50 %, 75 %).
- **IQR (Interquartile Range):** Rozdíl mezi 3. a 1. kvartilem – definuje „střední polovinu“ dat.
- `quantile()`: Výpočet libovolného percentilu.

```
# Get the 90th percentile (top 10 % of values)
top_threshold = df['revenue'].quantile(0.9)

# Filter rows above this threshold
top_performers = df[df['revenue'] > top_threshold]
```

Tvar rozložení a testování normality

Před použitím mnoha statistických metod (např. lineární regrese) musíme ověřit, zda mají data normální rozdělení.

- **Vizuální kontrola:** Histogram a Q-Q graf (Quantile-Quantile plot).
- **Šikmost (Skewness):** Míra asymetrie rozdělení (vlevo vs. vpravo).
- **Špičatost (Kurtosis):** Míra „špičatosti“ nebo „těžkých chvostů“ rozdělení.
- **Statistické testy:** Např. Shapiro-Wilkův test nebo D'Agostino's K^2 test.

```
# Calculate skewness (0 = normal distribution)
print(df['sales'].skew())

# Calculate kurtosis
print(df['sales'].kurt())
```

Ověření normality pomocí SciPy

Pandas skvěle spolupracuje s knihovnou `scipy.stats`, která obsahuje pokročilé testy.

```
from scipy import stats

# Testing the null hypothesis: "Data is normally distributed"
stat, p_value = stats.shapiro(df['sales'])

print(f"P-value: {p_value:.4f}")

if p_value > 0.05:
    print("Data vypadaji jako normalni rozdeleni.")
else:
    print("Data nemaji normalni rozdeleni (zamitame H0).")
```

Co dělat, když data nejsou normální?

Často pomůže transformace, například **logaritmická transformace** `np.log(df['sales'])`, která potlačí vliv extrémních hodnot a data „narovná“.

Identifikace a řešení outlierů

Odlehlé hodnoty mohou zásadně zkreslit průměr a směrodatnou odchylku.

- **Identifikace:** Pomocí Z-score nebo pravidla 1.5× IQR.
- **Řešení:**
  1. Odstranění (`drop`).
  2. Nahrazení (*Capping/Winsorizing*) – omezení na určitou mez.
  3. Ponechání (pokud jde o reálný, i když extrémní jev).

```
# Clip values to stay within a specific range
df['price_cleaned'] = df['price'].clip(lower=0, upper=df['price'].quantile(0.99))
```

Ověřování korelací

Korelace zkoumá lineární vztah mezi dvěma číselnými proměnnými. Výsledek se pohybuje v rozmezí -1 (nepřímá závislost) až $+1$ (přímá závislost).

```
# Calculate correlation between all numeric columns
# Note: Correlation does not imply causation!
corr_matrix = df.corr(numeric_only=True)

# Correlation between two specific columns
relationship = df['ad_spend'].corr(df['sales'])
```

Interpretace výsledku

- **0.7 až 1.0**: Silná závislost.
- **0.3 až 0.7**: Střední závislost.
- **0.0 až 0.3**: Slabá nebo žádná závislost.

Typy korelací v Pandas

Metoda `corr()` v Pandas podporuje tři hlavní typy pro měření závislosti:

- **Pearson (výchozí):** Měří *lineární* vztah. Vyžaduje normální rozdělení a citlivě reaguje na outliery.
- **Spearman:** Měří *monotónní* vztah pomocí pořadí (ranks). Je robustní vůči outlierům a vhodný pro nelineární data.
- **Kendall (Tau):** Podobně jako Spearman pracuje s pořadím, ale je vhodnější pro malé datasety a lépe zvládá shody v pořadí.

```
# Pearson correlation (linear)
pearson_corr = df['sales'].corr(df['marketing_spend'], method
    ='pearson')

# Spearman correlation (rank-based, non-linear)
spearman_corr = df['rank'].corr(df['satisfaction'], method='
    spearman')

# Matrix for all methods
full_matrix = df.corr(method='kendall')
```

Kdy kterou metodu zvolit?

Výběr závisí na charakteru vašich dat:

- **Pearson:** Použijte, pokud očekáváte přímou úměru (např. čím více metrů čtverečních, tím vyšší cena bytu).
- **Spearman/Kendall:** Použijte, pokud vztah roste, ale ne nutně lineárně (např. vliv věku na příjem), nebo pokud data obsahují extrémní hodnoty.

Praktický tip

Pokud se hodnoty Pearsonovy a Spearmanovy korelace výrazně liší, pravděpodobně máte v datech silné odlehlé hodnoty (outliery) nebo je vztah nelineární.

Výstupy a prezentace výsledků

Formátování tabulkových výstupů

Pro prezentaci v Jupyter Notebooku můžeme využít modul `df.style`, který umožňuje vizuálně zvýraznit klíčové informace podobně jako v Excelu.

- **Podmíněné formátování:** Zvýraznění maxim, minim nebo hodnot nad limit.
- **Barevné škály:** Vytvoření teplotních map (heatmaps) přímo v tabulce.
- **Formátování čísel:** Nastavení měn, procent nebo počtu desetinných míst bez trvalé změny dat.

```
# Highlighting max values and formatting as currency
styled_df = df.style.highlight_max(color='lightgreen') \
                    .format({'price': '{:.2f} CZK', 'margin':
                        '{:.1%}'})

# Visualizing values using bar charts inside cells
styled_df = df.style.bar(subset=['sales'], color='#d65f5f')

# Displaying the styled table
display(styled_df)
```

Podmíněné formátování napříč sloupci

Někdy potřebujeme nastavit styl buňky na základě hodnoty v jiném sloupci (např. obarvit název produktu, pokud je jeho marže záporná).

- Používáme metodu `df.style.apply()`.
- Definujeme funkci, která vrací matici CSS stylů o stejném rozměru jako původní DataFrame.
- Parametr `axis=1` zajistí, že funkce zpracovává řádky.

```
def highlight_by_status(row):
    styles = [''] * len(row)  # Default style
    if row['profit'] < 0:
        col_idx = row.index.get_loc('product_name')
        styles[col_idx] = 'background-color: #ffcccc; color: red'
    return styles

# Apply to the dataframe
styled_df = df.style.apply(highlight_by_status, axis=1)
display(styled_df)
```

Export do interaktivních a statických tabulek

Upravené tabulky lze následně exportovat pro použití v reportech nebo na webu.

```
# Export to HTML (retains all CSS styling)
html_output = df.style.background_gradient(cmap='viridis').
    to_html()
with open('report.html', 'w') as f:
    f.write(html_output)

# Export to LaTeX (for professional documents)
latex_code = df.to_latex(index=False, caption='Summary Table'
    , label='tab:summary')
```

Výhoda pro prezentaci

Zatímco `to_csv()` ukládá čistá data, `to_html()` nebo `to_latex()` umožňují sdílet výsledky i s vizuální interpretací (barvy, ohraničení).

Vizualizace jako nástroj komunikace

Grafy nejsou jen ozdobou, ale klíčovým nástrojem pro:

- **Identifikaci trendů:** Směřuje vývoj nahoru nebo dolů?
- **Srovnání kategorií:** Který segment je nejvýkonnější?
- **Hledání anomálií:** Existují v datech extrémní výkyvy?
- **Vysvětlení distribuce:** Jak jsou hodnoty rozprostřeny?

Knihovny v Pythonu

V kurzu se zaměříme na **Matplotlib** (základní stavební kámen) a jeho integraci přímo v Pandas.

Základní grafy v Pandas

Pandas nabízí metodu `plot()`, která automaticky využívá knihovnu Matplotlib.

```
import matplotlib.pyplot as plt

# Line plot (suitable for time series)
df.set_index('date')['sales'].plot(kind='line', title='Sales
    Trend')

# Bar plot (suitable for categories)
df.groupby('category')['revenue'].sum().plot(kind='bar')

plt.show() # Display the figure
```

Statistické grafy: Histogram a Boxplot

Pro hlubší analýzu rozdělení dat používáme specializované vizualizace.

- **Histogram:** Zobrazuje četnost hodnot v intervalech (bins).
- **Boxplot:** Zobrazuje medián, kvartily a odlehlé hodnoty (outliery).

```
# Histogram for price distribution
df['price'].plot(kind='hist', bins=20, title='Price
    Distribution')

# Boxplot for revenue by category
df.boxplot(column='revenue', by='category')
plt.xticks(rotation=45)
```

Vizuální ověření normality: Q-Q graf

Q-Q graf (Quantile-Quantile plot) srovnává kvantily vašich dat s kvantily teoretického normálního rozdělení.

- **Interpretace:** Pokud body leží přibližně na diagonální přímce, data mají normální rozdělení.
- **Odchylky:** Prohnutí bodů na koncích značí „těžké chvosty“ (outliery) nebo zkosení (skewness).

```
import scipy.stats as stats
import matplotlib.pyplot as plt

# Comparison of data against normal distribution
stats.probplot(df['sales'], dist="norm", plot=plt)
plt.title("Normal Q-Q Plot")
plt.show()
```

Odhad hustoty: KDE (Kernel Density Estimate)

Zatímco histogram je závislý na počtu intervalů (bins), **KDE graf** poskytuje hladkou křivku odhadu hustoty pravděpodobnosti.

- **Výhoda:** Lépe zobrazuje celkový „tvar“ dat a vícevrcholové (bimodální) distribuce.
- **Použití:** Ideální pro vizuální srovnání distribucí dvou skupin přes sebe.

```
# Standard density plot (KDE)
df['sales'].plot(kind='density', title='Kernel Density
    Estimation')

# Combination: Histogram with Density Curve
df['sales'].plot(kind='hist', density=True, alpha=0.5)
df['sales'].plot(kind='kde', color='red')
plt.show()
```

Kdy použít vizuální testy?

Statistické testy (jako Shapiro-Wilk) jsou u velkých datasetů často „přecitlivělé“ a zamítnou normalitu i při malých odchylkách.

Doporučení

Vždy kombinujte výpočet (p-hodnota) s vizuální kontrolou (Q-Q graf, Density plot). Pokud Q-Q graf vypadá lineárně, mírná odchylka v testu často nebrání použití parametrických metod.

Exportování výsledků

Hotovou analýzu a grafy potřebujeme uložit pro další použití v prezentacích nebo databázích.

```
# Save plot to high-resolution image
plt.savefig('analysis_result.png', dpi=300, bbox_inches='tight')

# Export cleaned data to CSV or Excel
df.to_csv('final_data.csv', index=False, sep=';')
df.to_excel('final_report.xlsx', sheet_name='Summary')

# Export to SQL database
from sqlalchemy import create_engine
engine = create_engine('sqlite:///results.db')
df.to_sql('marketing_analysis', engine, if_exists='replace')
```

Kam pokračovat dál

Shrnutí kurzu: Co jsme se naučili?

- **Práce v Jupyter Notebooku:** Efektivní psaní a dokumentace kódu.
- **Manipulace s daty:** Čištění, filtrování a transformace v Pandas.
- **Datová analytika:** Seskupování dat, pivot tabulky a statistika.
- **Vizualizace:** Převod čísel do srozumitelných grafů pomocí Matplotlib.

Zlaté pravidlo analytika

Data jsou jen tak dobrá, jak dobré otázky jim dokážete položit.

Ekosystém Python pro Data Science

Pandas je pouze základem širšího ekosystému:

- **NumPy:** Extrémně rychlé numerické výpočty (matice).
- **Seaborn:** Estetičtější a pokročilejší statistické grafy.
- **SciPy:** Pokročilá vědecká a technická matematika.
- **Plotly / Dash:** Interaktivní dashboardy pro web.

Úvod do Strojového učení (Machine Learning)

Knihovna **Scikit-learn** plynule navazuje na Pandas DataFrames.

```
from sklearn.linear_model import LinearRegression

# Pandas DataFrame to Machine Learning model
X = df[['feature_1', 'feature_2']] # Independent variables
y = df['target'] # What we want to predict

model = LinearRegression()
model.fit(X, y) # Training the model

predictions = model.predict(X)
```

Tipy pro další studium

- **Kaggle.com:** Platforma s reálnými datovými sadami a soutěžemi.
- **Oficiální dokumentace:** `pandas.pydata.org` (nejlepší zdroj informací).
- **Open Data:** Procvičování na veřejných datech (státní správa, meteorologie).
- **Kurz PYDATA2**: Pokročilá datová analýza.

Hodnocení

Budeme velmi rádi, pokud nám poskytnete hodnocení tohoto kurzu.

VSTUP PRO STUDENTY

Kód kurzu PYTH2

Jméno

Příjmení

Odeslat

ictPRO

Děkuji za pozornost!